Git: Control de Versiones para Equipos

Sistema que guarda el historial de tu código y permite trabajar en equipo sin conflictos

Made with GAMMA

¿Por qué Git es esencial?

Historial completo

Guarda "fotografías" del proyecto llamadas commits

Trabajo simultáneo

Múltiples personas programan al mismo tiempo

Volver atrás

Restaurar versiones anteriores si algo falla

Probar ideas

Experimentar sin afectar el proyecto principal

Made with GAMMA

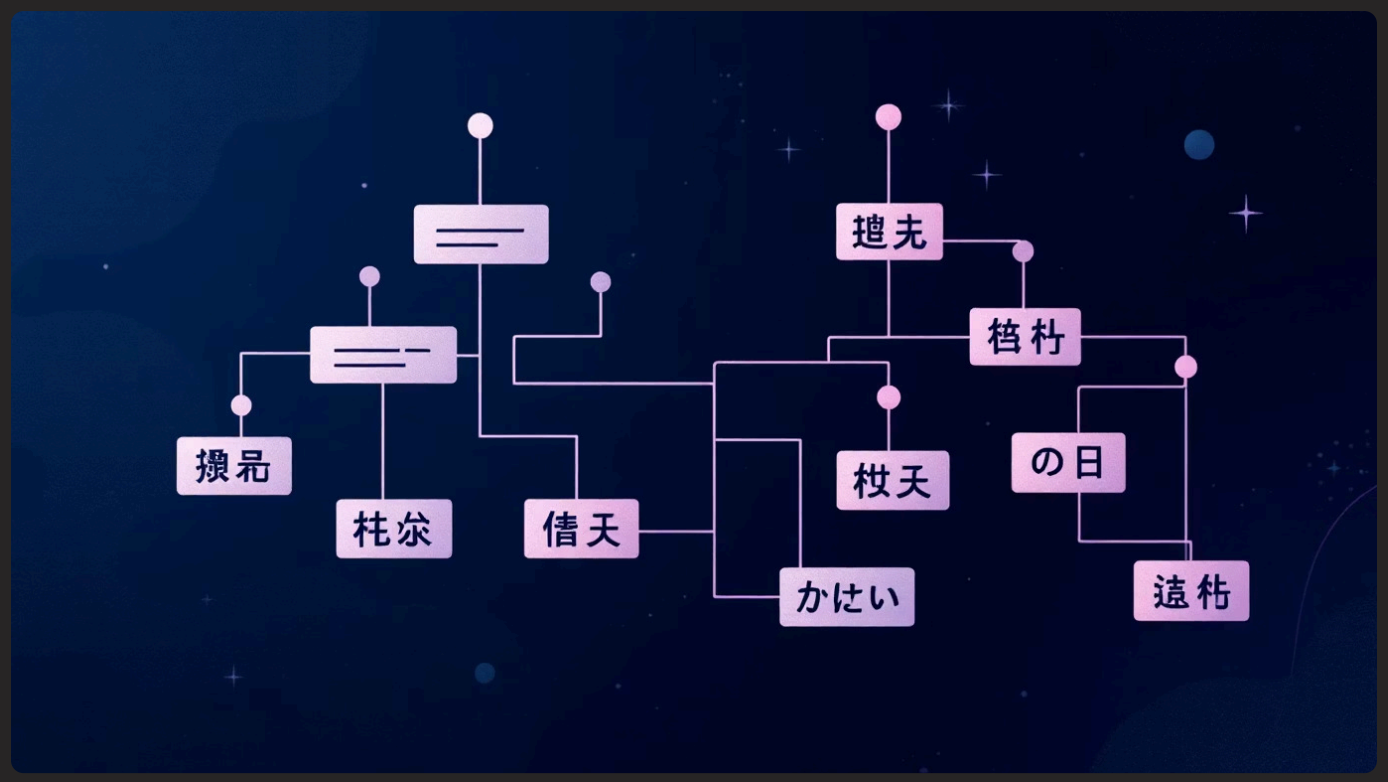
Flujo de trabajo en equipo

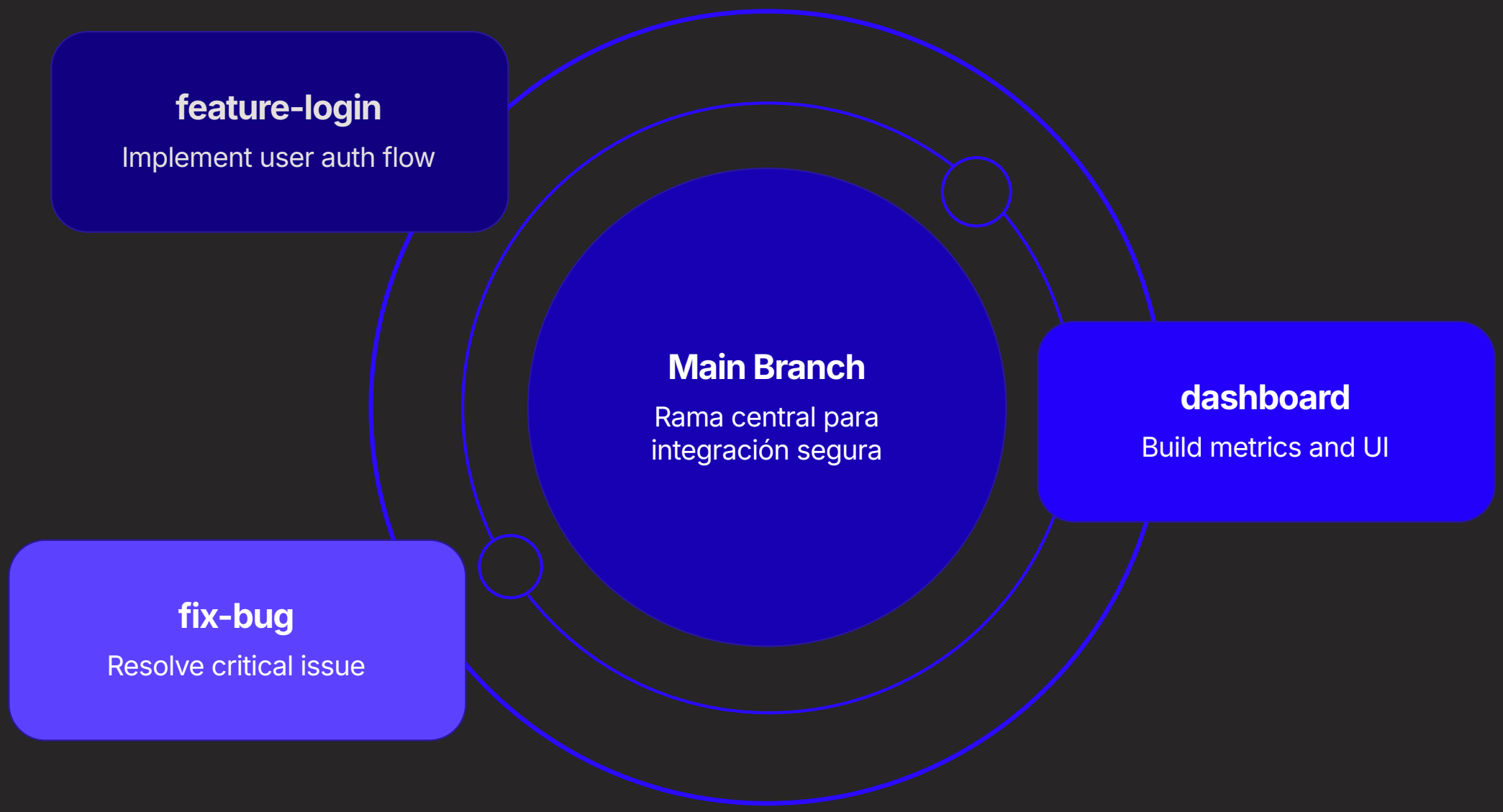
Ramas paralelas

Cada persona trabaja en su propia rama independiente

Integración segura

Los cambios se fusionan al proyecto principal cuando están listos





Made with GAMMA

Comandos básicos esenciales

01

Clonar proyecto

```
git clone URL
```

Descarga el repositorio por primera vez

02

Ver estado

```
git status
```

Muestra archivos modificados y listos

03

Preparar cambios

```
git add archivo
```

Agrega modificaciones al commit

04

Guardar cambios

```
git commit -m "mensaje"
```

Crea una "fotografía" del proyecto

05

Subir cambios

```
git push
```

Envía tus cambios al repositorio remoto

06

Descargar cambios

```
git pull
```

Obtiene las actualizaciones del equipo

Made with GAMMA

Trabajar con ramas

1

Crear rama

```
git checkout -b nombre
```

2

Cambiar rama

```
git checkout nombre
```

3

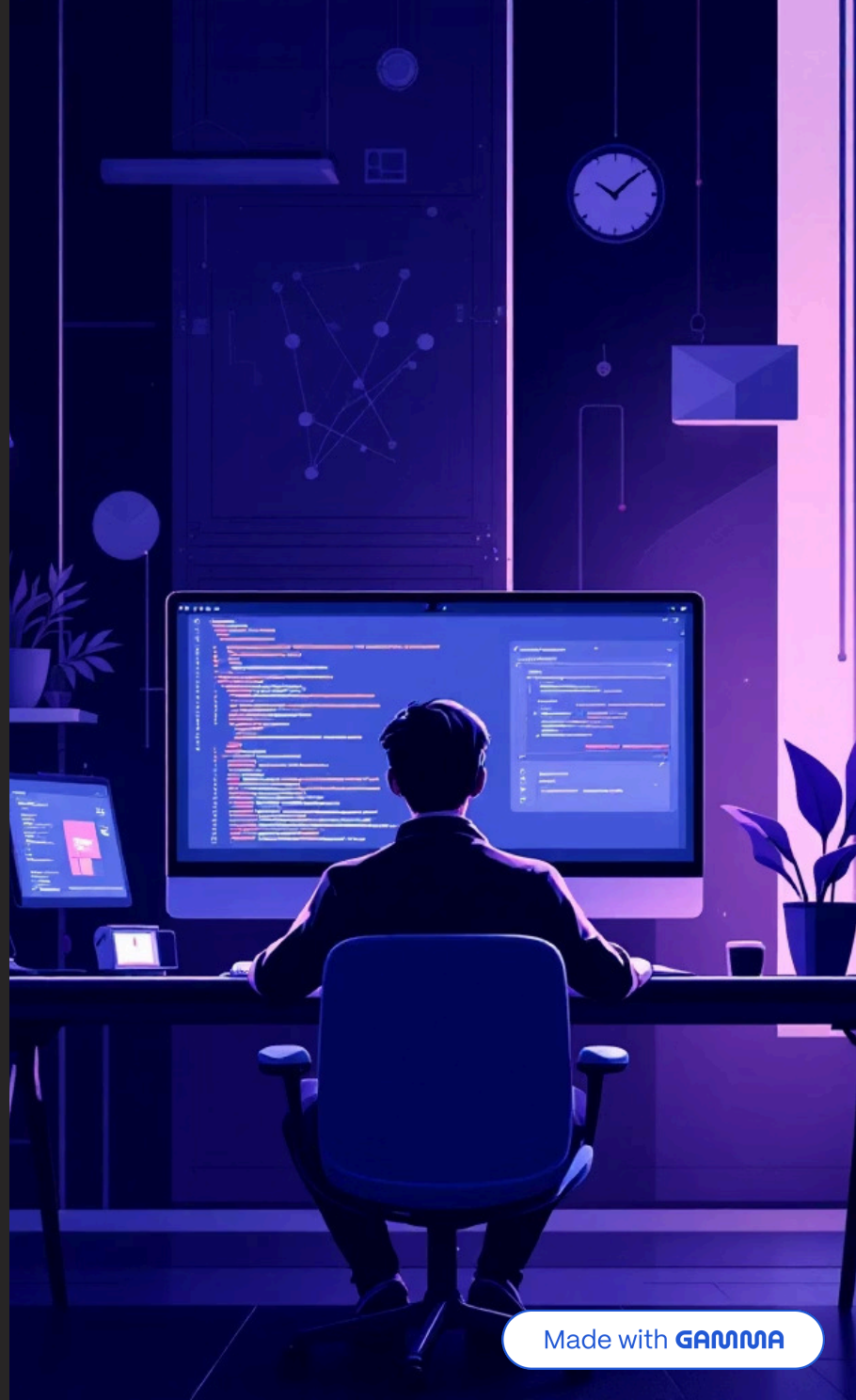
Listar ramas

```
git branch
```

4

Subir rama

```
git push origin nombre
```

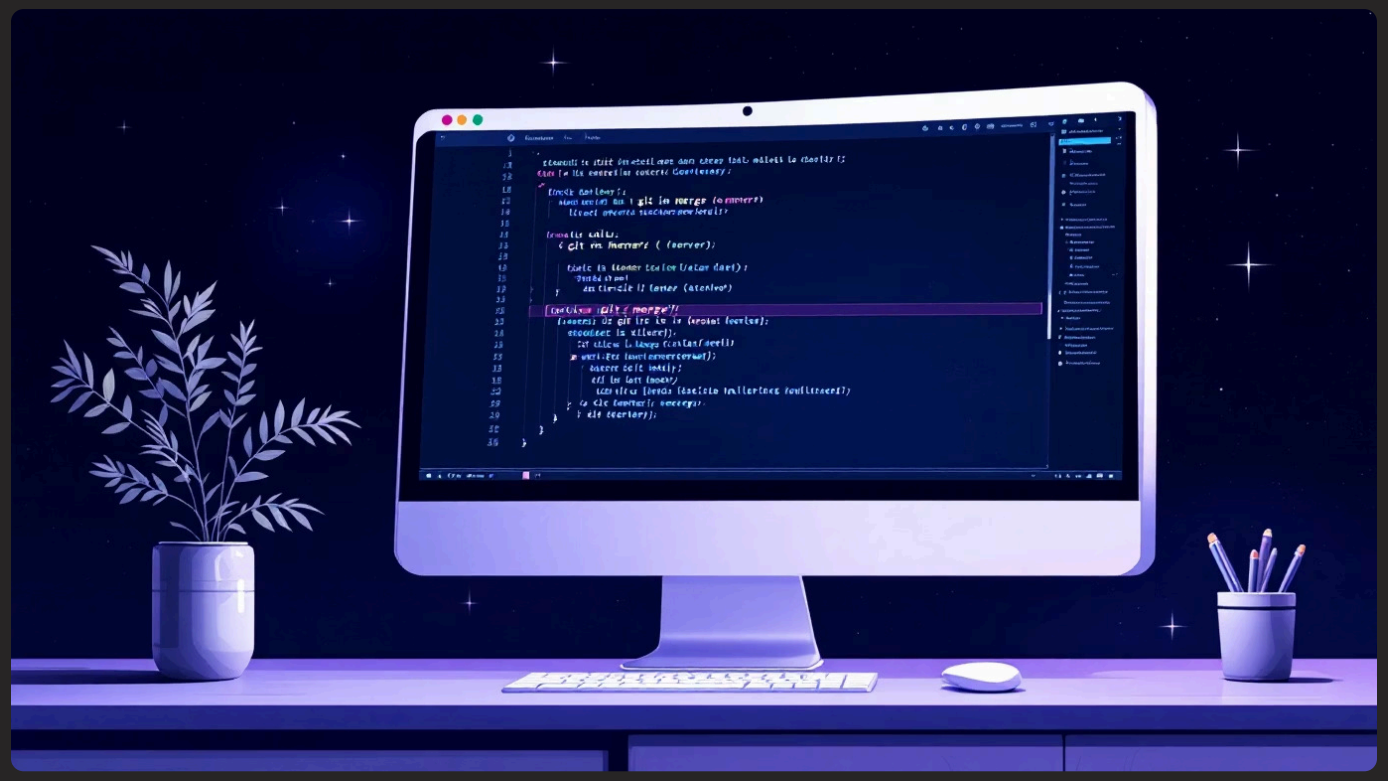


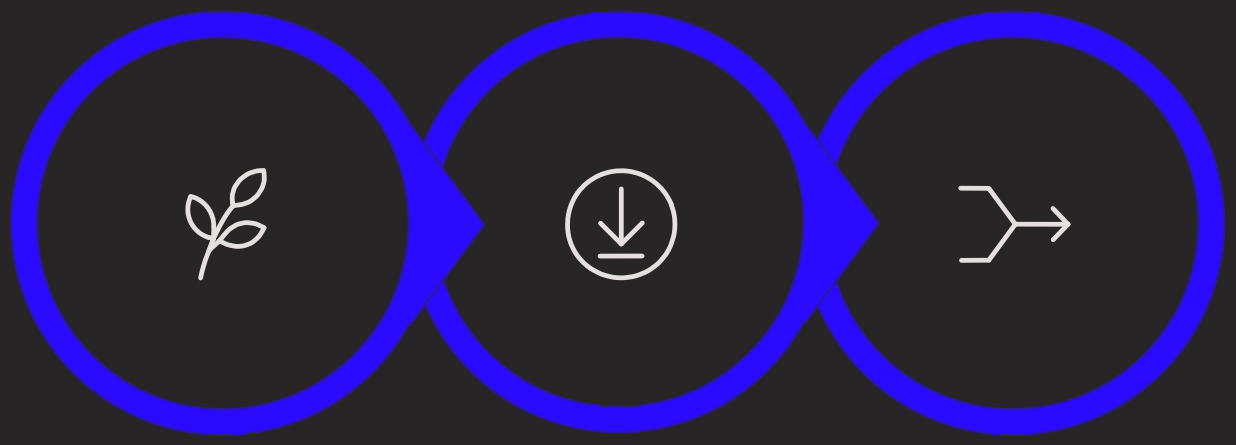
Made with GAMMA

Integrar cambios: El Merge

Pasos para integrar

1. Cambiar a main
2. Actualizar con pull
3. Fusionar la rama





Cambiar a main

Pull últimos

Fusionar rama

Made with GAMMA

Conflictos: Cuando dos personas editan lo mismo

¿Qué es un merge conflict?

Un conflicto ocurre cuando Git no puede decidir automáticamente qué cambios conservar. Esto sucede cuando dos personas han modificado la misma línea en el mismo archivo. Por ejemplo, si el desarrollador A y el desarrollador B editan ambos la línea 10 de `index.js`, Git detendrá la integración y pedirá una intervención manual.

¿Cómo se ve un conflicto en el código?

Git inserta marcadores especiales en el archivo afectado para que puedas identificar las versiones en conflicto:

```
<<<<<<< HEAD
código de tu rama actual
=======
código de la rama que quieres fusionar
>>>>>>> rama-destino
```

Cómo solucionarlo paso a paso

1. Git te avisa del conflicto durante `git merge`.
2. Revisa los archivos afectados usando `git status`.
3. Abre el archivo en tu editor de código preferido.
4. Decide qué código conservar (puedes elegir: solo A, solo B, o una combinación de ambos).
5. Borra los marcadores de conflicto (`<<<`, `===`, `>>>`).
6. Marca el archivo como resuelto con `git add <archivo>`.
7. Finaliza el proceso ejecutando `git commit`.

Ejemplo completo de comandos

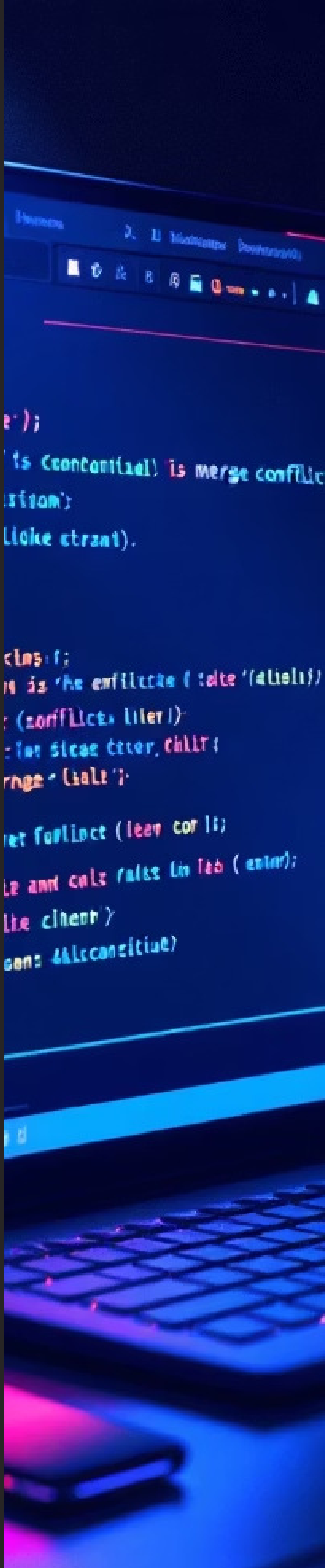
```
# 1. Intentar merge
git merge feature-branch
# (Git reporta conflicto)

# 2. Corregir archivos manualmente
# 3. Marcar como resuelto
git add index.js

# 4. Finalizar
git commit -m "Solucionar conflicto de merge"
```

Cómo evitar conflictos

- Haz `git pull` con frecuencia para mantener tu rama actualizada.
- Trabaja en ramas pequeñas y especializadas.
- Comunícate con tu equipo si vas a trabajar en archivos compartidos.
- Realiza cambios pequeños y frecuentes.

Tip extra: Los equipos profesionales suelen hacer un `git pull` y un `git merge` de la rama principal a su rama de trabajo antes de subir cambios (push) para resolver conflictos de forma local.

> "Un conflicto de merge no es un error de código, es Git pidiéndote permiso para decidir qué versión de la historia debe prevalecer."



Made with GAMMA

Errores que DEBES evitar

❌ Trabajar en main

Siempre crea una rama nueva para tus cambios

❌ Commits gigantes

Divide el trabajo en cambios pequeños y claros

❌ Sin probar código

Prueba siempre antes de subir cambios

❌ Olvidar git pull

Actualiza siempre antes de empezar a trabajar

❌ Archivos innecesarios

No subas node_modules, .env ni archivos de build

Made with GAMMA

Buenas prácticas

Mensajes descriptivos

fix: corrige error en validación

refactor: mejora estructura del login

feat: agrega nuevo componente

Ramas por feature

- feature-login
- feature-dashboard
- fix-login-bug

Un cambio = un commit

Cada commit debe representar una acción clara y específica

git pull al inicio

Siempre actualiza antes de comenzar tu trabajo diario

Made with GAMMA

Flujo simple para seguir

git pull

Actualiza con cambios del equipo

git checkout -b

Crea tu rama de trabajo

Trabaja

Implementa tus cambios

git add .

Prepara los cambios

git commit

Guarda con mensaje claro

git push

Sube al repositorio

Integra a main

Revisa y fusiona

> **Git no es solo guardar código. Es la herramienta que evita que el trabajo del equipo se rompa cuando todos programamos al mismo tiempo.**

Made with GAMMA